

Programmer dans la section Script

Guide des moyens de coder dans le panneau **Scripts** de l'application, avec exemples concrets. L'accent est mis sur l'exploitation des **movieclips** et des **variables au-delà des clips** : déclencher lecture/arrêt depuis un parent, un enfant, un script d'image, etc.

Sommaire

1. Le panneau Scripts : deux types de code
2. L'objet Scene (alias Game) — surface de l'API
3. Les éléments nommés : vos variables directes
4. Manipuler les propriétés d'un élément nommé
5. Les MovieClips : timeline indépendante
6. Piloter un clip enfant depuis le parent
7. Piloter le parent / la scène depuis un clip enfant
8. Les scripts d'image (frame actions)
9. Étiquettes d'image et gotoAndPlay('label')
10. Événements onEnterFrame, onKeyDown, onKeyUp
11. La map `named` — accès aux noms non-identifiants
12. Différences éditeur vs export HTML
13. Recettes courantes
14. Pièges et limites connues

1. Le panneau Scripts : deux types de code

Le panneau **Scripts** (icône `{ }`) gère deux familles de code distinctes, comme le navigateur d'actions d'Adobe Animate CC :

Type	Où il vit	Quand il s'exécute	Bouton / onglet
Script nommé	<code>doc.scripts[]</code> — onglets en haut du panneau	Quand on appuie sur Exécuter (ou Ctrl+Entrée) ; aussi une fois au chargement de l'export HTML	+ Nouveau, ► Exécuter
Script d'image (frame action)	<code>kf.script</code> sur une image clé précise	Automatiquement quand la tête de lecture atteint cette image <i>en lecture</i> (pas en scrub manuel)	+ Sur cette image

Les scripts d'image sont listés dans le sélecteur déroulant « *scripts d'image* » triés par calque et numéro d'image, pour sauter de l'un à l'autre. Un badge `a` sur la timeline ouvre directement le script d'une keyframe cliquée.

Note — Les deux types s'exécutent dans le même moteur (voir `src/runtime/sceneRuntime.js` → `execCode()`). Ils partagent donc les mêmes variables globales `Scene`, `Game`, `console`, `named`.

2. L'objet Scene (alias Game) — surface de l'API

`Scene` et `Game` sont deux noms pour le **même** objet. C'est la porte d'entrée vers la scène et sa timeline racine.

Propriétés de scène (lecture / écriture)

Propriété	Type	Description
<code>Scene.width</code>	number	Largeur de la scène (px). Réajuste le canvas à l'écriture.
<code>Scene.height</code>	number	Hauteur de la scène (px).
<code>Scene.frameRate</code>	number	Images par seconde (1-120). Défaut 24.
<code>Scene.backgroundColor</code>	string	Couleur de fond (ex. <code>"#ffffff"</code>).
<code>Scene.name</code>	string	Nom du document.
<code>Scene.frameCount</code>	number	Nombre total d'images du contexte courant (scène racine ou symbole en édition).

Lecture / navigation dans la timeline

Méthode / prop.	Description
<code>Scene.play()</code>	Lance la lecture de la timeline racine.
<code>Scene.stop()</code>	Arrête la timeline racine.
<code>Scene.gotoAndPlay(frame)</code>	Saute à l'image <code>frame</code> (0-indexé) et joue.
<code>Scene.gotoAndStop(frame)</code>	Saute à l'image <code>frame</code> et s'arrête.
<code>Scene.playing</code> <code>get</code>	<code>true</code> si la lecture est en cours.
<code>Scene.currentFrame</code> <code>get/set</code>	Image courante (0-indexé). Modifiable directement.

Création d'éléments, boucle et entrées

Méthode	Description
<code>Scene.addShape(type, props)</code>	Ajoute une forme ("rect" , "ellipse" , "line" , "path" , "text") sur l'image clé courante du calque actif. Retourne l'id.
<code>Scene.addInstance(symbolId, props)</code>	Ajoute une instance de symbole (movieclip/graphic). Retourne l'id.
<code>Scene.onEnterFrame(cb)</code>	Enregistre <code>cb(frame)</code> appelée à chaque image pendant la lecture.
<code>Scene.onKeyDown(cb)</code>	Enregistre <code>cb(key, event)</code> sur appui clavier.
<code>Scene.onKeyUp(cb)</code>	Idem au relâchement.
<code>Scene.keys</code> get	Objet { [key]: true/false } — état clavier en temps réel.
<code>Scene.random(n)</code>	Entier aléatoire dans [0, n[.
<code>Scene.log(...args)</code>	Affiche dans la console du panneau (et la console navigateur).

```
// Exemple : configurer la scène et démarrer
Scene.frameRate = 30;
Scene.backgroundColor = "#102030";
Scene.frameCount = 60;
Scene.log("Scène :", Scene.width, "x", Scene.height);
Scene.play();
```

Note — Dans un **script d'image d'un clip enfant**, ce même objet `Scene` est **scopé au clip** (voir §5) : `Scene.stop()` arrête alors *ce clip*, pas la racine. La distinction est automatique, vous n'avez rien à changer dans votre code.

3. Les éléments nommés : vos variables directes

C'est la pierre angulaire pour piloter un clip « par-delà lui-même ». Dans le **Panneau Propriétés**, le champ « *Nom d'instance* » donne un nom à un élément. Tout élément nommé présent à l'image courante est alors injecté comme **variable globale** dans chaque script :

```
// Si un movieclip porte le Nom d'instance "hero" :
hero.x += 2;           // déplacement horizontal
hero.rotation = 45;    // rotation en degrés
hero.gotoAndStop(5);    // si c'est un movieclip → contrôle sa timeline
```

L'injection se fait au moment de l'exécution par un préambule généré automatiquement (voir `sceneRuntime.js` → `execCode()`) :

```
var hero = named["hero"];
var fond = named["fond"];
// ... votre code ...
```

Note — Si deux éléments portent le même nom, le **calque supérieur gagne** (comportement Flash). La résolution se fait à l'image courante du contexte d'édition.

4. Manipuler les propriétés d'un élément nommé

Les propriétés animables exposées via la variable directe :

Propriété	Type	Sens
<code>x</code> , <code>y</code>	number	Position (px), origine = coin haut-gauche de la scène
<code>rotation</code>	number	Rotation en degrés (sens horaire)
<code>scaleX</code> , <code>scaleY</code>	number	Échelle (1 = 100%)
<code>opacity</code>	number 0-1	Opacité
<code>width</code> , <code>height</code>	number	Dimensions (rect, ellipse, text, bitmap, instance)
<code>fill</code> , <code>stroke</code>	string	Couleurs (formes)
<code>strokeWidth</code>	number	Épaisseur du trait
<code>text</code> , <code>fontSize</code>	string / number	Texte et taille (formes de type text)
<code>name</code>	string	Nom d'instance (lecture/écriture)

```
// Osciller un clip nommé "balle" avec onEnterFrame
Scene.onEnterFrame(function(frame) {
    balle.x = 200 + Math.sin(frame * 0.2) * 100;
    balle.y = 150 + Math.cos(frame * 0.2) * 60;
    balle.rotation = frame * 5;
});
```

5. Les MovieClips : timeline indépendante

Un symbole de type **movieclip** possède sa **propre timeline indépendante** : `isPlaying`, `currentFrame`, `play/stop/gotoAndPlay/gotoAndStop`. C'est exactement le modèle d'Adobe Animate. Chaque **instance** de movieclip a son propre état de lecture, indexé par l'id d'instance (pas par l'id de symbole — deux placements du même clip jouent indépendamment).

Dans l'éditeur, l'état de lecture des clips est géré par `src/runtime/clipStates.js` : un `Map<instanceId, ClipState>` où chaque entrée vaut `{ isPlaying: true, currentFrame: 0, _lastScriptFrame: -1, loop:`

`true` } . À l'export, la classe `MovieClip` (`src/export/tweenRuntime.js`) porte ces états directement sur l'instance.

Quand un movieclip est nommé, sa variable expose en plus

Méthode / prop.	Effet sur ce clip
<code>nom.play()</code>	Lance la lecture du clip
<code>nom.stop()</code>	Arrête le clip à l'image courante
<code>nom.gotoAndPlay(f)</code>	Saute à l'image <code>f</code> (ou une étiquette) et joue
<code>nom.gotoAndStop(f)</code>	Saute à l'image <code>f</code> et s'arrête
<code>nom.currentFrame</code>	Image courante du clip (0-indexé)
<code>nom.frameCount</code>	Nombre d'images du symbole
<code>nom.isPlaying</code> éditeur export	<code>true</code> si le clip joue
<code>nom.loop</code> export	Boucle activée (défaut <code>true</code>)
<code>nom.addEventListener('loop' 'complete', fn)</code> export	Événements de fin/boucle du clip

Note — Les méthodes de timeline sur la variable nommée ne marchent **que pour les instances de movieclip**. Un symbole de type `graphic` reste synchronisé sur l'image du parent (pas d'état propre), comme dans l'éditeur.

6. Piloter un clip enfant depuis le parent

Cas le plus fréquent : sur la scène racine (ou dans un clip parent) se trouve une instance de movieclip nommée `porte`. On veut la commander depuis un script du parent.

```
// Script racine : la porte s'ouvre quand on appuie sur Espace
Scene.stop(); // on gèle la timeline racine, on pilote les clips à la main

Scene.onKeyDown(function(key) {
  if (key === " " || key === "Spacebar") {
    porte.gotoAndPlay("ouverture"); // par étiquette (voir §9)
  }
});

Scene.onKeyUp(function(key) {
  if (key === " " || key === "Spacebar") {
    porte.gotoAndStop("fermee");
  }
});
```

```
// Autre exemple : démarrer/arrêter deux clips enfants en alternance
clipA.play();
clipB.stop();

Scene.onEnterFrame(function(f) {
    if (f % 60 === 0) { // toutes les 60 images
        if (clipA.isPlaying) { clipA.stop(); clipB.play(); }
        else { clipA.play(); clipB.stop(); }
    }
});
```

Dans l'éditeur, l'avancement des clips enfants est fait par `main.js` → `advanceClipsForLayer()` à chaque image de la timeline racine, **après** exécution des scripts d'image du clip (comportement Animate : le script de l'image courante peut appeler `Scene.stop()` et empêcher l'avancement).

7. Piloter le parent / la scène depuis un clip enfant

C'est l'inverse : un script d'image à l'intérieur d'un movieclip veut agir sur le clip qui le contient, ou sur la scène racine. Le mécanisme repose sur le **Scene scopé** :

- À l'intérieur d'un clip enfant, `Scene.stop()` arrête **ce clip enfant**, pas le parent ni la racine.
- Pour commander le parent ou la racine, on passe par le **Nom d'instance** du parent tel qu'il est connu du contexte parent. Concrètement, on nomme l'instance du parent (ex. `monClip`) et depuis l'enfant, on ne peut pas y accéder directement — la résolution des noms se fait *dans le contexte du script qui s'exécute*.

Attention — Il n'y a pas de `this.parent` explicite comme en ActionScript. La communication parent ⇌ enfant se fait toujours par **noms d'instance** exposés au contexte qui exécute le script.

Recette : un clip enfant arrête la scène racine

1. Donnez à l'instance du clip parent sur la scène racine le Nom d'instance `hero`.
2. Sur l'image clé où vous voulez stopper la scène, créez un **script d'image à la racine** :

```
// Script d'image à la racine, image 20
Scene.stop();
hero.gotoAndStop("victoire");
```

3. À l'intérieur de `hero`, sur l'image de fin de l'animation `"victoire"`, posez un script d'image **du clip** :

```
// Script d'image DANS le clip hero, image "victoire"
Scene.stop(); // arrête hero (Scene est scopé au clip ici)
// Pas d'accès direct à la racine depuis ici ; la coordination se fait
// par convention d'étiquettes / d'images, comme dans Animate CC.
```

Recette : un clip enfant signale la fin au parent

À l'export, `MovieClip` émet l'événement `'complete'` quand le clip termine sans boucler. Le parent peut donc écouter l'enfant nommé :

```
// Script racine
hero.loop = false;
hero.addEventListener("complete", function() {
    Scene.gotoAndPlay("niveauSuivant");
});
```

Note — `addEventListener / removeEventListener` et l'événement `'complete'` ne sont disponibles que dans le runtime d'**export** (`tweenRuntime.js`). Dans l'éditeur, on reproduit ce schéma avec un `onEnterFrame` qui teste `hero.currentFrame === hero.frameCount - 1`.

8. Les scripts d'image (frame actions)

Un script d'image est rattaché à une **image clé** d'un **calque** précis. Il s'exécute **une seule fois** quand la tête de lecture atteint cette image *pendant la lecture* (jamais en scrub manuel, comme dans Animate CC).

Créer un script d'image

1. Sélectionnez le calque et placez la tête de lecture sur l'image voulue.
2. Insérez une image clé (F6) si nécessaire.
3. Dans le panneau Scripts, cliquez sur **+ Sur cette image**. Un squelette est créé :

```
// Exécuté une fois quand la tête de lecture atteint cette image
Scene.stop();
```

Comportement clip vs racine

Emplacement du script d'image	<code>Scene.stop()</code> arrête...
Calque de la scène racine	la timeline racine
Calque d'un symbole movieclip en édition	ce clip (état via <code>clipState</code>)
Calque d'un symbole graphic	sans effet — un graphic n'a pas d'état propre

Dans l'éditeur, le déclenchement est dans `main.js` → `advanceClipsForLayer()` : on exécute le script de l'image courante du clip **avant** d'avancer, pour qu'un `Scene.stop()` puisse empêcher l'avancement.

```
// Script d'image à la racine, image 0 : stop + lancement d'un clip
Scene.stop();
intro.gotoAndPlay(1);
```

```
// Script d'image dans un clip "jauge", image 10
// Ici Scene est scopé au clip jaugue
Scene.gotoAndStop("plein");
```

9. Étiquettes d'image et gotoAndPlay('label')

Le panneau Propriétés permet d'affecter une **étiquette** à une image (via `frameLabels` du document ou du symbole). On peut alors cibler une image par son nom plutôt que par son numéro — aussi bien sur la scène qu'un clip :

```
// À la racine
Scene.gotoAndPlay("intro");

// Sur une instance nommée "bouton" (movieclip)
bouton.gotoAndStop("survol");

// Dans un script d'image du clip bouton lui-même
Scene.gotoAndPlay("clac");
```

À l'export, les étiquettes sont inversées en `{ [label]: frameIndex }` pour un lookup $O(1)$ (voir `model.js` → `invertFrameLabels()`). Un label introuvable produit un avertissement console et ne fait rien.

10. Événements onEnterFrame, onKeyDown, onKeyUp

Les callbacks enregistrés via `Scene.onEnterFrame(cb)` / `onKeyDown` / `onKeyUp` sont **vidés à chaque Exécuter** (voir `sceneRuntime.js` → `run()`) pour éviter les accumulateurs d'un run à l'autre. Les `onKey*` sont ignorés quand le focus est dans un champ de saisie.

```
// Déplacement au clavier d'un clip "joueur"
Scene.onKeyDown(function(key) {
    if (key === "ArrowLeft") joueur.x -= 4;
    if (key === "ArrowRight") joueur.x += 4;
    if (key === "ArrowUp") joueur.y -= 4;
    if (key === "ArrowDown") joueur.y += 4;
});

// État clavier en temps réel (utile dans onEnterFrame)
Scene.onEnterFrame(function() {
    if (Scene.keys["ArrowRight"]) joueur.x += 3;
    if (Scene.keys[" "]) joueur.gotoAndPlay("saut");
});
```

```
// Détection de collision frame par frame entre deux clips nommés
Scene.onEnterFrame(function(f) {
    var dx = balle.x - cible.x;
    var dy = balle.y - cible.y;
    if (Math.hypot(dx, dy) < 30) {
        cible.gotoAndStop("touche");
        Scene.stop();
    }
});
```

11. La map `named` — accès aux noms non-identifiants

Un Nom d'instance qui n'est pas un identifiant JavaScript valide (espace, accent, mot réservé, commence par un chiffre...) ne peut pas être injecté comme variable directe — ce serait une `SyntaxError`. Il reste accessible via la **map** `named` passée en argument implicite à tout script :

```
// Nom d'instance = "1er plan" (espace + chiffre)
named["1er plan"].x += 1;
named["1er plan"].opacity = 0.8;
```

Les mots réservés (`Scene`, `Game`, `console`, `eval`, `this`, `var` ...) sont aussi exclus de l'injection directe (voir la liste `RESERVED` dans `sceneRuntime.js`). Utilisez `named["var"]` en pareil cas.

12. Différences éditeur vs export HTML

La même surface d'API est reproduite à l'export (voir `exportHTML.js` → `buildBootstrapScript()`), mais quelques nuances subsistent :

Aspect	Éditeur	Export HTML
Moteur de scripts	<code>sceneRuntime.js</code> + <code>clipStates.js</code>	Classe <code>MovieClip</code> de <code>tweenRuntime.js</code>
État d'un clip enfant nommé	<code>clipState.isPlaying</code> via Map	propriétés directes du <code>MovieClip</code> enfant
<code>nom.isPlaying</code> / <code>nom.loop</code>	<code>isPlaying</code> dispo	les deux dispo
<code>addEventListener('complete' 'loop')</code>	non exposé	dispo sur les clips
Création d'éléments (<code>addShape</code> / <code>addInstance</code>)	dispo	dispo (ajoute à la première keyframe visible non verrouillée)
Variable directe <code>nom.method()</code> sur movieclip	via l'objet élément + <code>runClipFrameScripts</code>	via un <code>Proxy</code> qui délègue au <code>MovieClip</code> enfant
Scène scopée pour un script d'image de clip	<code>runClipFrameScripts</code> construit un <code>clipScene</code>	<code>_runChildFrameScript</code> + <code>_buildClipScene</code>

Note — L'export **n'inclut pas** la dépendance Konva : le rendu est fait par le runtime canvas 2D pur de `tweenRuntime.js`, importé tel quel (un seul fichier source pour l'éditeur et l'export).

13. Recettes courantes

Bouton play/stop sur un clip

```
// Clip nommé "anim" sur la scène
Scene.onKeyDown(function(k) {
  if (k === "p") { anim.isPlaying ? anim.stop() : anim.play(); }
});
```

Changer d'animation sur un personnage

```
// "hero" a plusieurs étiquettes : "idle", "walk", "jump"
Scene.onKeyDown(function(k) {
  if (k === "ArrowRight") hero.gotoAndPlay("walk");
  if (k === " ")          hero.gotoAndPlay("jump");
});
Scene.onKeyUp(function(k) {
  if (k === "ArrowRight") hero.gotoAndStop("idle");
});
```

Synchroniser deux clips (un attend la fin de l'autre)

```
// clipA fait 30 images ; on lance clipB quand clipA atteint la fin
Scene.stop();
clipA.play();
Scene.onEnterFrame(function() {
  if (clipA.currentFrame === clipA.frameCount - 1) {
    clipB.gotoAndPlay(0);
  }
});
```

Compteur de boucles sur un clip (export)

```
var tours = 0;
roue.loop = true;
roue.addEventListener("loop", function() {
  tours++;
  Scene.log("Tours :", tours);
  if (tours >= 5) roue.stop();
});
```

Stopper la scène à la dernière image d'un clip enfant

```
// Script d'image à l'intérieur du clip "fin", dernière image
Scene.stop();           // arrête ce clip
// Pour arrêter aussi la racine : passer par un script d'image racine
// qui surveille fin.currentFrame (voir recette ci-dessus)
```

14. Pièges et limites connues

- **Pas de `this.parent`**. La communication parent $\rightleftharpoons$ enfant se fait uniquement par Noms d'instance, résolus dans le contexte du script qui s'exécute. Un script d'un clip enfant ne voit pas les noms du parent.
- **Les `graphic` n'ont pas d'état propre**. Un symbole `graphic` imbriqué suit l'image du parent (modulo `frameCount`) ; `play` / `stop` n'ont aucun effet sur lui.
- **Callbacks vidés à chaque Exécuter**. Relancer un script nommé efface les `onEnterFrame` / `onKey*` précédents — pas d'accumulation.
- **Scripts d'image en lecture seulement**. Le scrub manuel de la timeline n'exécute pas les scripts d'image (comme Animate CC).
- **Noms réservés**. Un nom d'instance égal à `Scene`, `Game`, `console`, `named` ou un mot-clé JS n'est pas injecté comme variable : passez par `named["Scene"]`.
- **Deux instances, deux états**. L'indexation se fait par `instanceId`, pas par `symbolId` — deux placements du même clip jouent indépendamment.
- **`addEventListener` à l'export uniquement**. Dans l'éditeur, on reproduit la logique avec `onEnterFrame` + tests de `currentFrame` / `isPlaying`.

Sources principales : `src/ui/ScriptsPanel.js`, `src/runtime/sceneRuntime.js`, `src/runtime/clipStates.js`, `src/main.js` (boucle de lecture), `src/core/model.js` (`getNamedElements`, `symbols`), `src/export/tweenRuntime.js` (classe `MovieClip`), `src/export/exportHTML.js` (bootstrap d'export).

Généré pour TweenJS / Animate-JS v0.1 — Septembre 2026.